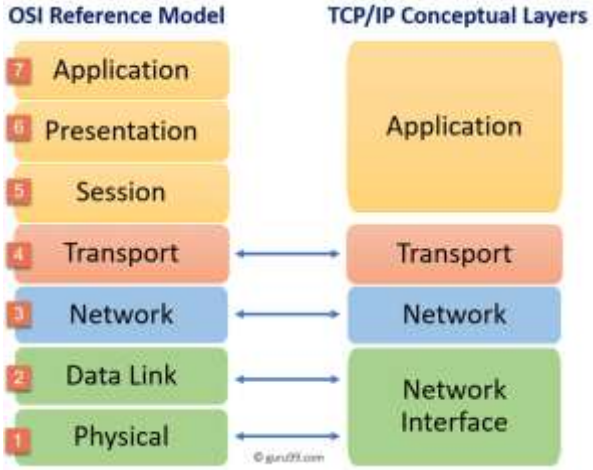
بـخش $Network$

۱ – مدل TCP/IP برای ما نحوه ی اتصـــال یک کامپیوتر به اینترنت و چگونگی ارســـال دیتا بین آنها را مشـخص می کند. همچنین به ما کمک می کند تا یک شـبکه ی مجازی را هنگامی که چندین کامپیوتر به یک دیگر متصـــل شـــده اند، ایجاد نماییم. همانگونه که از نام این مدل پیداســـت، این مدل مبتنی بر $TCP(Transmission\ Control\ Protocol)$ و $IP(Internet\ Porotocol)$ اســـت. این مدل به طورخاص به عنوان مدلی برای ارسال با ضریب اطمینان بالا و استریم $end-to-end\ byte$ در یک شبکه ی نامطمئن طراحی شده است.

مدل OSI یا $Open\ System\ Interconnection$، یک مدل منطقی و نظری برای تعریف نحوه ی ارتباط در شبکه ا ست که سیستم ها از آن برای برقراری ارتباط و اتصال با سیستمهای دیگر درون شبکه استفاده می کنند. همچنین این مدل با استفاده از پروتکلهای موجود در لایه های مختلف ، روشی مؤثر برای انتقال $packet$ ها را برای ما مشخص می سازد.

در شکل ۱ یک شمای کلی از لایه های موجود در هرکدام از مدل ها را می بینید.



شکل ۱ لایه های مختلف هریک از مدل های TCP/IP و OSI

حال در ادامه به ذکر تفاوت های این دو مدل می پردازیم:

- همانطوری که در شکل ۱ می بینید، $OSI\ model$ دارای ۷ لایه و $TCP\ /\ IP\ model$ شامل ۴ لایه است. در مدل OSI، بهتر است هر لایه در زمانی که تعداد محدودی از لایه های $abstraction$ لازم است، ایجاد گردد. از طرفی تعداد لایه ها باید به حدی زیاد باشد که تابع های مستقل از هم در یک لایه قرار نگیرند و در عین حال باید به اندازه‌ای کوچک باشد تا معماری شبکه بسیار پیچیده نگردد و البته بعضی از سرویس ها در لایه های مختلف تکرار می شوند. این مدل به راحتی با هر نوع از پروتکل های جدید سازگار می شود البته در این مدل از پروتکل خاصی استفاده نمی شود و در مقابل $TCP\ /\ IP\ model$ ، تمامی پروتکل های مسیریابی را پشتیبانی می کند. همچنین در این مدل هر لایه برای اجرای نقش اصلی خود مبتنی بر لایه ی زیرین است و از این جهت لایه های پایین تر باید بتوانند به لایه ی بالاتر بعدی سرویس ارائه کنند و تغییرات در یک لایه

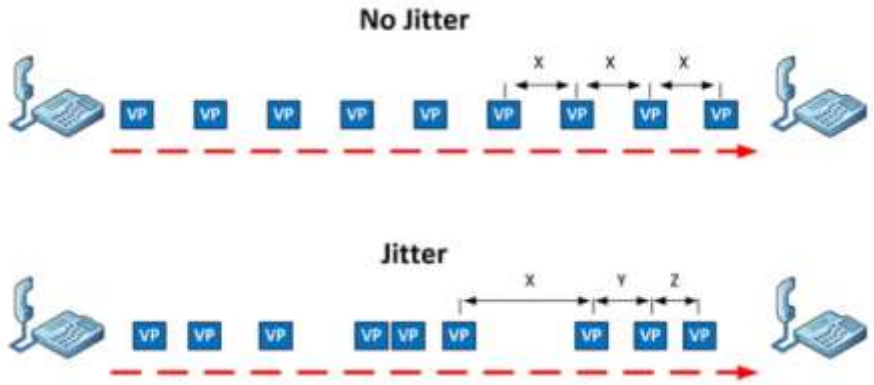
نباید موجب ایجاد تغییر در سایر لایه ها شود. مدل TCP / IP از معماری های انعطاف پذیر پشتیبانی می کند و افزودن سیستمهای بیشتر به شبکه با استفاده از این مدل برای ما راحت تر است.

- همانطوری که در تعریف بالا دیدید، سیستم ها از $OSI\ model$ برای بازکردن اتصال و یا ارتباط بین سیستمی استفاده می کنند، در واقع $OSI\ model$ در همگون سازی تمامی انواع سخت‌افزار اعم از رتور، سوئیچ و $motherboard$ به ما کمک می کند، درمقابل $TCP / IP\ model$ به ما کمک می کند تا نحوه ی اتصال هر نوع از کامپیوتر به شبکه و نیز انتقال دیتا میان آنها را مستقل از سیستم عامل آندو مشخص سازیم و برای برقراری ارتباط میان دو کامپیوتر می توانیم از آن استفاده کنیم. در مدل TCP / IP، شبکه تا زمانی که ارتباط بین $source$ و $destination$ به طور مناسب برقرار است، دست نخورده باقی می ماند و نیز تضمین می کند تا بسته هایی که خارج از توالی معمول ارسال بسته ها دریافت شده اند را در جایگاه اصلی شان قرار دهد. این مدل به ما اجازه می دهد تا نرخ دیتای ارسالی را کنترل کنیم و بنابراین هیچگاه فرستنده بسته هایی با حجمی بیشتر از توان دریافتی گیرنده ارسال نمی کند.
- سایز هدر در مدل OSI، ۵ بایت و در مدل TCP / IP برابر با ۲۰ بایت است.
- $transport\ layer$ در $OSI\ model$، $connection - oriented$ است ولی در $TCP / IP\ model$ می تواند هم $connection - oriented$ و یا $connectiomless$ باشد.
- در $OSI\ model$ به طور واضح، میان سرویسها، پروتکل ها و $interface$ ها، تمایز وجود دارد، در حالیکه در TCP / IP چنین نیست.
- برای تعریف کردن استانداردهای مسیریابی و پروتکلها، $OSI\ model$ از $network\ layer$ و $TCP / IP\ model$ تنها از $Internet\ layer$ استفاده می کند.
- با توجه به به ساختار لایه ها و نحوه ی ارتباط آنها با یکدیگر که قبلاً به آن اشاره کردیم، $OSI\ model$ برای برقراری ارتباط $vertical\ approach$ رو دنبال می کند ولایه ها به صورت موازی قادر به کارکردن نمی باشند زیرا هر لایه برای ارائه ی سرویس نیاز به دست یابی به داده از لایه های پایین تر دارد. در حالیکه $TCP / IP\ model$ از $horizonal\ approach$ استفاده می کند. همانطور که پیش تر نیز به آن پرداختیم $TCP / IP\ model$ انعطاف پذیرتر از $OSI\ model$ است. در واقع اینجا هر گره اجازه ی تصمیم گیری بین کاری که خودش خواهان انجام آن است و کار محول شده به آن را دارد و همانطوری که قبلاً گفتیم، هیچگاه فرستنده بسته هایی با حجم بیشتر از توان دریافتی گیرنده، به گونه ای که گیرنده قادر به پاسخگویی نباشد را ارسال نمی کند.
- همان گونه که در شکل ۱ ملاحظه کردید، لایه های $Data\ Link$ و $Physical$ در مدل OSI از یکدیگر جدا هستند ولی در مدل TCP / IP این دو لایه در لایه ای با نام $host - to - network$ در هم ادغام شده اند.

- همانطوری که در شکل ۱ مشاهده می شود، $OSI\ model$ دارای دو لایه به نام های $session$ و $presentation$ است و این دو لایه در $TCP\ /\ IP\ model$ وجود ندارند و به عبارتی پیاده سازی این دو لایه در این مدل به $developer$ ها واگذار شده است.
- و درآخر $TCP\ /\ IP\ model$ قبل از ظهور اینترنت و $OSI\ model$ پس از ظهور اینترنت تعریف شده اند.
- افزودن یک دستگاه جدید در $TCP\ /\ IP\ model$ بسیار ساده است و از این جهت $TCP\ /\ IP\ model$ از مقیاس پذیری بالایی برخوردار است.

و در آخر همان گونه که گفته شد $OSI\ model$ یک مدل تئوری است و تا به حال پیاده سازی نشده است و در حال حاضر از $TCP\ /\ IP\ model$ استفاده می شود.

۲ - در واقع $jitter$ اختلاف زمانی بین لحظه ی ارسال $packet$ درشبکه تا لحظه ی دریافت آن در مقصد است. از طرفی دیگر تأخیر یا $latency$ مدت زمانیست که این $datapacket$ در شبکه می ماند تا به مقصد برسد، اما با توجه به شرایط شبکه (به عنوان مثال کیفیت پایین اتصال در شبکه های وایرلس یا عدم وجود پهنای باند کافی برای پشتیبانی از تعداد بالای کاربران ($Network\ Congestion$) و یا عدم الویت بندی بسته ها در شبکه های $VoIP$) یا گاهی براثر کارایی پایین سخت افزار، $datapacket$ ها پشت سر هم و به ترتیبی که باید در مقصد دریافت نمی شوند و از طرفی ممکن است برخی از $datapacket$ ها با تأخیر به مقصد برسند. همانطور که قبلا گفته شد با توجه به شرایط کلی شبکه و اتصال در آن لحظه، تاخیر رخ می دهد و اکثراً هیچ مشکلی هم ایجاد نمی کند. (زیرا این گونه تغییرات در حدود چند میلی ثانیه هستند.) اما در برخی از شبکه های مخابراتی همین چندین میلی ثانیه نیز می تواند دردسرآفرین باشد، برای ارسال صوت و تصویر میزان تأخیر باید از ۱۰ میلی ثانیه کمتر باشد اما برای ارسال متن تا ۳۰ میلی ثانیه تأخیر نیز قابل قبول است. به عنوان مثال برای انتقال فریم های صوتی برای ارتباطات $VoIP$ اگر تاخیر از ۱۰ میلی ثانیه بیشتر شود بر کیفیت اتصال اثر سو گذاشته و ممکنست که صدا پس و پیش شنیده شود، در اینجا $jitter$ به عنوان یک عنصر مشکل ساز معرفی می شود و در این حالت پروتکل $TCP\ /\ IP$ با تأثیر $jitter$ مقابله می کند. در شکل ۲ قرارگیری بسته ها و فواصل زمانی میان آنها را در ارسال همراه با $jitter$ و بدون آن را مشاهده می کنید.



شکل ۲ قرارگیری ترتیب بسته ها در ارسال همراه با $jitter$ و بدون آن

زمانی که می خواهیم از گره A به گره B پیام ارسال کنیم، اطلاعات به فرم $data\ packet$ و در فواصل زمانی معینی ارسال می شوند. حال بسته به شرایط شبکه و نوع پیام ارسالی ممکنست مدت ارسال متفاوت باشد. این چندگونگی زمانی برای ارسال $data\ packet$ ها، $jitter$ یا $Packet\ Delay\ Variation$ نام دارد و نباید آن را با $delay$ اشتباه گرفت. حال برای کاهش میزان $jitter$ در شبکه های $VoIP$ از $jitter\ buffer$ استفاده می کنیم. این بافر وظیفه ی تنظیم سرعت را برعهده دارد و در ابتدا یک برآورد از حداقل و حداکثر میزان $jitter$ برای هر $data\ packet$ انجام می دهد و علاوه بر این یک میانگین از $jitter$ ها محاسبه می کند و آن بسته را در بافر قرار می دهد هرچقدر این اندازه بیشتر شود متناسب با آن کیفیت کاهش می یابد و در شبکه ها با میزان $jitter$ بالا، ممکنست استفاده از $jitter\ buffer$ کارآمد نباشد. البته از سوی دیگر قرار نیست که همه ی $data\ packet$ های ارسالی با میزان تأخیر بالا در بافر قرار بگیرند. گاهی اوقات لازم است که این $data\ packet$، $drop$ شوند.

حال در ادامه به توضیح درباره ی $ping$ می پردازیم.

$ping$ ، مدت زمانیست که طول می کشد تا یک $data\ packet$ از یک $device$ به سرور رفته و دوباره از سرور به آن $device$ برگردد. پینگ درواقع تأخیر زمانی است که در هنگام انجام هر کار در شبکه رخ می دهد. از $ping$ تنها برای برای اندازه‌گیری میزان تأخیر استفاده نمی شود. هر شبکه ای از تعداد زیادی $device$ و سرور تشکیل‌شده است. هرکدام از این $device$ ها می‌توانند به $device$ موجود در مقصد، $packet$ ارسال کنند.$data\ packet$ ارسالی توسط $device$ مقصد دریافت شده و برای آن دسته از $data\ packet$ ها که نیاز به ارسال $response$ دارند در جواب یک $data\ packet$ به $device$ فرستنده ارسال می‌کند.

این مراحل برای هر $ping$ تکرار می‌شود. $ping$ این امکان را به ما می دهد تا مدت زمان رفت و برگشت $data\ packet$ ها بین $device$ ها را اندازه گیری کنیم. استفاده از دستور $ping$ برای چک کردن وضعیت سرورها کمک می کند. به عنوان مثال اگر یکی از سرورها $down$ شود و یا اگر سرعت آن کم شود، می‌تواند از برقراری اتصال ممانعت کند. در این وضیعت$packet\ loss$ رخ می دهد یعنی $device$ فرستنده یک $data\ packet$ ارسال می کند ولی $device$ گیرنده آن را دریافت نمی کند. همچنین با استفاده از $ping$ می توان $data\ packet$ های از دست رفته را مشاهده نمود. به عنوان مثال اگر پس از اجرای دستور $ping$ با پیام $Request\ timed\ out$ روبرو شدید، این بدین معناست که $Packet$ توسط گیرنده در مقصد دریافت نشده است و در طول مسیر ارسال و دریافت $packet\ loss$ رخ داده است و یا سرور به فرستنده $response$ ارسال نکرده است. به طور معمول این تأخیر برای اینترنت پرسرعت بین ۵ تا ۳۰ میلی ثانیه است.

۳ – در شکل ۳ نحوه ی استفاده از $ping$ برای سرور گوگل را در cmd و $data\ packet$ های ارسال شده و دریافت شده را می بینید و مقدار متوسط تأخیر برای دستگاه من ۶۰ میلی ثانیه است و منظور از TLL، $time\ to\ live$ است و به معنای زمان رسیدن پیام پس از لحظه ی ارسال تا زمان دریافت، هنگام انتقال بین دو یا چند سیستم است.

Command Prompt

```
Microsoft Windows [Version 10.0.19042.1237]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Taban>ping google.com

Pinging google.com [172.217.18.142] with 32 bytes of data:
Reply from 172.217.18.142: bytes=32 time=78ms TTL=47
Reply from 172.217.18.142: bytes=32 time=59ms TTL=47
Reply from 172.217.18.142: bytes=32 time=53ms TTL=47
Reply from 172.217.18.142: bytes=32 time=52ms TTL=47

Ping statistics for 172.217.18.142:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 52ms, Maximum = 78ms, Average = 60ms
```

شکل ۳ نحوه ی اجرای ping در cmd

۴ –

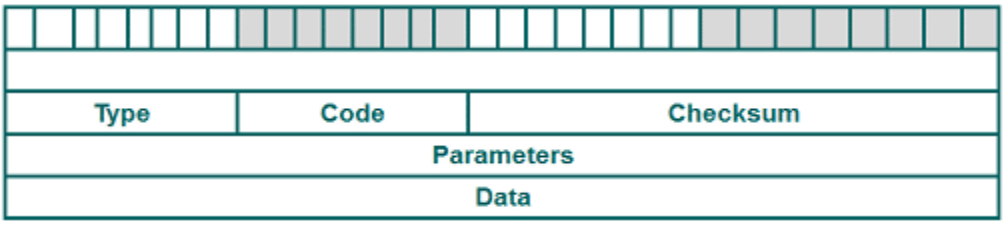
$Telnet$: $telnet$ یک پروتکل $client - server$ است. در واقع کامپیوترها برای اتصال با یک‌دیگر، از زبان خاصی استفاده می کنند. $telnet$ که در اصل یک پروتکل محسوب می شود، یکی از این زبانها است. این عبارت مخفف کلمه ی $Teletype\ Network$ است و همان گونه که از نام این پروتکل پیداست می توانیم به وسیله ی آن و از طریق cmd، یک دستگاه را از راه دور (مستقل از سیستم عامل آن) کنترل کنیم. در این پروسه، کامپیوتری که می خواهد به کامپیوتر دیگری در شبکه متصل شود، اصطلاحاً $client$ نام می‌گیرد. معمولاً دستگاهی که قرار است اتصال به کمک آن انجام شود یک سرور است، دستگاه مقصد هم اصطلاحاً $host$ نام دارد. تلنت برخلاف بیشتر پروتکل‌های موجود در شبکه مانند $HTTP$، علاوه بر ارسال دیتا، امکان دسترسی به فایل‌های مختلف در $host$ را برای $client$ (بدون حضور فیزیکی پشت کامپیوتر $host$) فراهم می‌کند و $client$ می‌تواند فایل‌های موجود در میزبان را تغییر و حتی برنامه های موجود در $host$ را اجرا نماید. همچنین در برخی از مواقع متخصصین شبکه برای راه‌اندازی اولیه سخت‌افزارها از $Telnet$ استفاده می کنند. علاوه بر این از $Telnet$ برای پینگ گرفتن از یک پورت به‌منظور بررسی باز یا بسته بودن آن استفاده می شود. پس از وارد کردن دستور مربوطه، اگر پورت باز باشد با یک صفحه خالی مواجه می شویم و اگر پورت بسته باشد پیغام خطا نشان داده می‌شود. در حال حاضر استفاده از $Telnet$ کاهش چشمگیری پیدا کرده است اما این پروتکل هنوز منسوخ نشده و به جای آن در برخی از موارد از SSH استفاده می شود و این پروتکل بر پورت ۲۳ قرار گرفته است.

RDP : RDP یا $Remote\ Desktop\ Protocol$ یک پروتکل برای اتصال از راه دور یک کامپیوتر به کامپیوتر دیگر تحت سیستم عامل ویندوز است که این پروتکل توسط شرکت مایکروسافت ارائه شده است. در واقع این پروتکل یک رابط گرافیکی یا GUI برای اتصال به یک کامپیوتر دیگر از طریق شبکه فراهم می‌کند، این رابط باید به گونه ای باشد که با استفاده از آن در حالت تمام صفحه تقریباً هیچ تفاوتی با استفاده ی مستقیم از کامپیوتر مقصد وجود نداشته باشد. در واقع یک $client$ در معماری RDP می‌تواند به سرور

متصل شود تا بدون حضور فیزیکی به کامپیوتر مقصد دسترسی یابد. این پروتکل بر پورت ۳۳۸۹ قرار گرفته است.

$IMAP$: $IMAP$ مخفف $Internet\ Message\ Access\ Protocol$ است که به عنوان یک پروتکل برای دسترسی به پیغام در اینترنت تعریف شده است. البته سرویس های $host$ ایمیل، از پروتکل های $IMAP$ و $POP3$ و $SMTP$ استفاده می کنند. پروتکل $IMAP$ در $Application\ layer$ و بر روی پورت ۱۴۳ قرار دارد و به گیرنده ها اجازه دسترسی به ایمیل بر روی سرویس دهنده را از طریق کنترل از راه دور می دهد. برای مثال سرویس های $Electronic\ Mail\ Box$ و یا وب سایت هایی که در آنها فایلهای صوتی و تصویری بصورت آنلاین استفاده می شوند از این پروتکل بهره گرفته اند. تفاوت عمده ی میان $POP3$ و $IMAP$ در نوع ارتباط با سرور است. در $POP3$ ، $client$ ایمیل های جدید را بر روی کامپیوتر خود دانلود می کند و می تواند بدون ارتباط مستقیم با سرور، ایمیل ها را مطالعه کند، اما در پروتکل $IMAP$ ، ارتباط با سرور به صورت مستقیم است و ایمیل ها، برای $client$ به صورت بی وقفه ارسال می گردند. همچنین اگر ایمیلی توسط کاربر خوانده شود، در صورت برقراری ارتباط با استفاده از $IMAP$ ، ایمیل مربوطه در وب میل و برای سایر $client$ هایی که به آن ایمیل دسترسی دارند، خوانده شده تلقی می شود. قابلیت همزمان سازی در $IMAP$ برای کاربرانی که از چند کامپیوتر استفاده می کنند، بسیار کاربردی است و این قابلیت در $POP3$ وجود ندارد. البته برخی از ISP ها به خوبی از $IMAP$ پشتیبانی نمی کنند.

$ICMP$: پروتکل $ICMP$ یا $Internet\ Control\ Message\ Protocol$ است که به طور کلی برای تشخیص خطا در هنگام بروز و ارسال آن به گره فرستنده استفاده می شود که بررسی کلی وضعیت اتصال بین روتر وسرور و یا بررسی وجود یا عدم وجود سیگنال و خطایابی در کامپیوترها، روترها و هاست را نیز شامل می شود. این پروتکل در لایه ی $network$ قرار گرفته ولی مانند پروتکل های لایه ی $transport$ رفتار می کند و شماره ی پورت مربوط به این پروتکل، ۷ است. در شکل ۴ فیلد های مربوط به پیام های $ICMP$ را می بینید. در واقع طول پیام ها برابر با ۳۲ بیت است و هر پیام دارای سه فیلد با نام های $type$، $code$ و $checkesum$ است.



شکل ۴ نمونه ای از پیام در پروتکل icmp

عددی که در فیلد $type$ قرار می گیرد بیانگر نوع پیام می باشد و ساختار فیلدهای $data$ و $parameters$، بسته به عددی که در این فیلد قرار می گیرد ، متفاوت خواهد بود. در فیلد $code$ گاهی خود پیغام به چند زیرنوع دیگر تقسیم می شود که کد زیرنوع در این فیلد قرار می گیرد و در آخر از محتوای

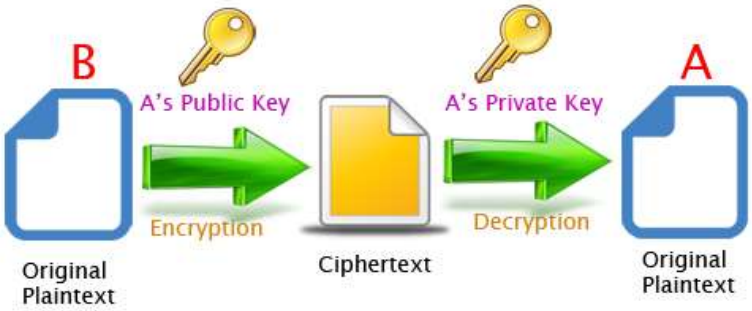
فیلد $checksum$ برای سنجش اعتبار و سلامت بسته استفاده می شود. تمام بسته $ICMP$ به صورت دو بایت دوبایت جمع شده و نهایتاً عددی ۱۶ بیتی به دست می آید که درون این فیلد قرار می گیرد.

۵ – ابتدا برای توضیح کدگذاری نامتقارن، به چند تعریف دیگر می پردازیم:

$private\ key$: در این حالت برای $Encryption$ و $Decryption$ پیام ها، اصطلاحاً از کلید خصوصی که یک کلید مخفی است استفاده می کنیم. $private\ key$ همراه با $public\ key$ استفاده می شود و در تمام زمانها باید به صورت خصوصی نگه داشته شود و از افشای آن باید پرهیز کرد. در ارزهای رمزنگاری شده ، از $private\ key$ برای محافظت از دارایی ها توسط کیف پول استفاده می شود. مثلاً هر کیف پول $Bitcoin$ دارای یک $private\ key$ است که برای ارتباط با آن باید $private\ key$ مربوطه را در دست داشت. در واقع زمان ایجاد کیف پول $public\ key$ و $private\ key$ همراه آن تولید می شوند و توجه داشته باشید که اگر $private\ key$ کیف پول خود را گم کنید ، دیگر نمی توانید به محتوای آن دسترسی داشته باشید.

$public\ key$: از $public\ key$، تنها برای $Encryption$ پیام استفاده می شود و ارسال آن برای افراد دیگر مجاز است. هنگامی که فرستنده پیام را با استفاده از $public\ key$ رمز گذاری کرد، تنها با استفاده از $private\ key$ خود می توانید قفل آن پیام را باز کنید.

رمزنگاری نامتقارن از رمزنگاری متقارن کمی پیچیده تر است. تمایز اصلی آن با رمزنگاری متقارن استفاده ی آن از هر دو نوع کلید $private\ key$ و $public\ key$ است به عبارتی دیگر در رمزنگاری نامتقارن به جای استفاده از یک کلید مشترک، از جفت کلید (یک $public\ key$ و $private\ key$) برای $Encryption$ و $Decryption$ داده ها، استفاده می شود. در شکل ۵ خلاصه ی آنچه که در بالا گفته شده را می بینید.



شکل ۵ استفاده از public key و private key در رمزنگاری نامتقارن

به عنوان مثال اگر یک $public\ key$ را به عنوان یک $username$ در نظر بگیریم، $password$ برای این کاربر یک $private\ key$ محسوب می شود که در دسترس عموم نیست و نباید آن را با کسی به اشتراک بگذارد. به عبارت دیگر هر $username$ با یک $password$ گره خورده است. $username$ قابل اشتراک گذاری است و همه با در دست داشتن آن می توانند سابقه حساب را ببینند ولی در مقابل راهی برای یافتن $password$ از روی $username$ وجود ندارد. یک $private\ key$ می تواند به عنوان $password$ یک کاربر خاص (یا $username$ متناسب با آن) دیده شود که دسترسی به حساب و انجام پردازش های موردنظر بدون در دست داشتن آن امکان پذیر نیست.

در ابتدا فرستنده ، پیام را با یک $public\ key$ گیرنده رمزگذاری می کند . حال فرستنده مجاز به ارسال پیام است، زیرا تنها راه برای رمز گشایی و مشاهده ی پیام استفاده ی از $private\ key$ مربوط به آن است که فقط گیرنده دارد . حال گیرنده پس از دریافت پیام با استفاده از $private\ key$ مرتبط، رمز گشایی را انجام می دهد. استفاده از جفت کلید در رمز نگاری نامتقارن آن را نسبت به رمزگذاری متقارن به روشی امن تری برای رمزنگاری تبدیل می کند.

۶ - $Secure\ Socket\ Layer$ یا به اختصار SSL برای برقراری ارتباط امن میان سرویس دهنده و سرویس گیرنده در اینترنت استفاده می شود. از این پروتکل برای امن کردن پروتکل های غیرامن مثل $HTTP$ و یا $IMAP$ استفاده می شود. در واقع تعدادی از الگوریتم های رمزنگاری بر روی داده های خام که قرار است از یک کانال ارتباطی غیرامن مثل اینترنت عبور کنند، اعمال می‌شود تا ایمن ماندن داده را در طول انتقال تضمین می‌کند. SSL در واقع پروتکلی است که در آن ارتباطات با استفاده یک کلید، رمزگذاری می شوند. زمانی که قرار است یکسری اطلاعات را به صورت ایمن به گیرنده ای که گواهی نامه ی SSL را دارد ارسال شود، مراحل زیر انجام می شوند:

وقتی سروری بخواهد پروتکل SSL را فعال کند. ابتدا یک $public\ key$ می سازد.

سپس $public\ key$ را همراه با یک درخواست گواهی نامه ی SSL به یکی از صادرکنندگان این گواهی نامه ها مثل $verisign$ می فرستد.

$verisign$ نیز ابتدا مشخصات و میزان قابل اعتماد بودن و امنیت سرور را بررسی می کند و $public\ key$ را دوباره رمزگذاری می کند و آن را برای سرور می فرستد تا برای انتقال امن اطلاعات خود از آن استفاده کند. که به کلید جدید ساخته شده $private\ key$ می گوییم.

حال هر زمان که یک کاربر بخواهد از طریق پروتکل SSL به اطلاعات دسترسی داشته باشد، ابتدا کامپیوتر کاربر یک $public\ key$ برای سرور می فرستد.

سرور نیز این $public\ key$ را با کلید امنیتی خود مخلوط کرده و از آن کلید جدیدی می سازد. سپس آن را به کامپیوتر کاربر می فرستد.

از این به بعد تمامی اطلاعاتی که بین کاربر و سرور جابجا می شوند با این کلید جدید رمز گذاری می شوند.

در واقع شرکتی که برای صدور و اعطاء گواهی های دیجیتال SSL صلاحیت دارد، برای هر کدام از دو طرفی که قرار است اتصال میان شبکه‌ای امن میان آنها برقرار شود، گواهی‌های مخصوص سرویس دهنده و سرویس گیرنده را صادر کرده و با مکانیزم های احراز هویت خاص خود، هویت هر کدام از طرفین را برای طرف مقابل تأیید می‌کند و به هرکدام از آنها یک گواهینامه بر اساس هویت آنها به منظور دانستن هویت طرفین ارتباط، نشانی ها و تعیین هویت هرکدام برای طرف مقابل ... اعطا می کند. علاوه بر این باید تضمین کند تا اگر اطلاعات حین ارسال سرقت شد، برای رباینده غیر قابل درک باشد. که رمزگذاری ها و رمز گشایی های لازم به کمک الگوریتم های رمزنگاری نامتقارن و متقارن انجام می شود.